

DecodeLabs Industrial Training Kit

Project 3

SQL Data Analysis

Extracting Business Insights Using Structured Query Language

Batch: 2026 | Powered by DecodeLabs

1. Project Objective

This project demonstrates the practical use of SQL to extract, filter, group, and aggregate data from a relational dataset. The goal is to translate raw transactional records into actionable business intelligence using declarative queries. All queries adhere to the SQL logical execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

2. Tools & Environment

- **Database:** SQLite / MySQL (ANSI SQL compatible)
- **Client:** DB Browser for SQLite / MySQL Workbench
- **Skills applied:** SELECT, WHERE, ORDER BY, GROUP BY, HAVING, COUNT, SUM, AVG, MIN, MAX, LIKE, aliases.

3. Dataset — Company Sales & Employees

Two tables were created to simulate a small business scenario: an **employees** table (interns and staff across departments) and a **sales** table (transactions closed by each employee).

Schema (DDL):

```
CREATE TABLE employees (  
  emp_id      INT PRIMARY KEY,  
  name        VARCHAR(50),  
  role        VARCHAR(30),  
  department  VARCHAR(30),  
  salary      DECIMAL(10,2),  
  email       VARCHAR(80),  
  join_date   DATE  
);  
  
CREATE TABLE sales (  
  sale_id     INT PRIMARY KEY,  
  emp_id      INT,  
  product     VARCHAR(40),  
  category    VARCHAR(30),  
  amount      DECIMAL(10,2),  
  sale_date   DATE,  
  FOREIGN KEY (emp_id) REFERENCES employees(emp_id)  
);
```

employees (sample rows):

emp_id	name	role	department	salary	join_date
1	Aarav Sharma	Developer	Tech	65000	2024-01-15
2	Diya Patel	Marketer	Marketing	48000	2023-11-02
3	Rohan Verma	Developer	Tech	72000	2022-06-10
4	Isha Gupta	Designer	Design	55000	2024-03-21
5	Kabir Singh	Analyst	Tech	60000	2023-08-14
6	Meera Nair	Marketer	Marketing	52000	2024-02-05
7	Vikram Rao	Designer	Design	58000	2022-12-19
8	Ananya Iyer	Developer	Tech	80000	2021-09-30

sales (sample rows):

sale_id	emp_id	product	category	amount	sale_date
101	1	Website Build	Service	15000	2025-01-10
102	3	Mobile App	Service	28000	2025-01-18
103	2	Ad Campaign	Marketing	9000	2025-02-05
104	4	UI Kit	Design	6500	2025-02-11
105	1	Website Build	Service	17000	2025-02-22
106	5	Data Report	Analytics	12000	2025-03-03
107	6	SEO Package	Marketing	8000	2025-03-15
108	8	API Platform	Service	42000	2025-03-20
109	7	Brand Identity	Design	11000	2025-04-02
110	3	Mobile App	Service	31000	2025-04-19
111	2	Ad Campaign	Marketing	9500	2025-05-06
112	8	API Platform	Service	45000	2025-05-25

4. SQL Queries & Analysis

Q1. Basic SELECT — list all employees

Purpose: Retrieve every employee record to understand the workforce.

```
SELECT emp_id, name, role, department, salary
FROM employees;
```

Result:

emp_id	name	role	department	salary
1	Aarav Sharma	Developer	Tech	65000
2	Diya Patel	Marketer	Marketing	48000
3	Rohan Verma	Developer	Tech	72000
4	Isha Gupta	Designer	Design	55000
5	Kabir Singh	Analyst	Tech	60000
6	Meera Nair	Marketer	Marketing	52000
7	Vikram Rao	Designer	Design	58000
8	Ananya Iyer	Developer	Tech	80000

Insight: The company employs 8 people spread across Tech, Marketing and Design — Tech being the largest function.

Q2. WHERE — filter high earners

Purpose: Isolate employees whose salary is at or above Rs. 60,000.

```
SELECT name, role, salary
FROM employees
WHERE salary >= 60000
ORDER BY salary DESC;
```

Result:

name	role	salary
Ananya Iyer	Developer	80000
Rohan Verma	Developer	72000
Aarav Sharma	Developer	65000
Kabir Singh	Analyst	60000

Insight: All top earners belong to the Tech department, confirming that engineering talent commands the highest compensation.

Q3. Pattern matching with LIKE

Purpose: Find every developer using a role-based text pattern.

```
SELECT name, department
FROM employees
WHERE role LIKE 'Dev%';
```

Result:

name	department
Aarav Sharma	Tech
Rohan Verma	Tech
Ananya Iyer	Tech

Insight: Three developers form the core of the Tech department.

Q4. GROUP BY + COUNT — headcount per department

Purpose: Collapse rows into departmental buckets and count members.

```
SELECT department, COUNT(*) AS headcount
FROM employees
GROUP BY department
ORDER BY headcount DESC;
```

Result:

department	headcount
Tech	4
Marketing	2
Design	2

Insight: Tech is twice the size of the other departments — indicating a product-heavy company structure.

Q5. Aggregation — average salary per department

Purpose: Use AVG() to reveal pay norms across departments.

```
SELECT department,
       ROUND(AVG(salary), 2) AS avg_salary,
       MIN(salary)           AS min_salary,
       MAX(salary)           AS max_salary
FROM employees
GROUP BY department;
```

Result:

department	avg_salary	min_salary	max_salary
Tech	69250.0	60000	80000
Marketing	50000.0	48000	52000
Design	56500.0	55000	58000

Insight: Tech offers the highest average salary (Rs. 69,250) with the widest spread — Marketing is the most compressed pay band.

Q6. SUM — total revenue per employee

Purpose: Join sales to employees and sum revenue closed by each person.

```
SELECT e.name,
       COUNT(s.sale_id) AS deals,
       SUM(s.amount)    AS total_revenue
FROM employees e
JOIN sales s ON s.emp_id = e.emp_id
```

```
GROUP BY e.name
ORDER BY total_revenue DESC;
```

Result:

name	deals	total_revenue
Ananya Iyer	2	87000
Rohan Verma	2	59000
Aarav Sharma	2	32000
Diya Patel	2	18500
Kabir Singh	1	12000
Vikram Rao	1	11000
Meera Nair	1	8000
Isha Gupta	1	6500

Insight: Ananya Iyer is the top revenue contributor, closing Rs. 87,000 — nearly 5× the marketing average.

Q7. GROUP BY category — revenue mix

Purpose: Understand which product category drives the business.

```
SELECT category,
       COUNT(*)      AS deals,
       SUM(amount)    AS revenue,
       ROUND(AVG(amount),2) AS avg_deal
FROM sales
GROUP BY category
ORDER BY revenue DESC;
```

Result:

category	deals	revenue	avg_deal
Service	6	178000	29666.67
Marketing	3	26500	8833.33
Design	2	17500	8750.0
Analytics	1	12000	12000.0

Insight: Services dominate revenue (~76% of total). Marketing has volume but small deal size — a candidate for upselling.

Q8. HAVING — filter aggregated groups

Purpose: Show only categories whose total revenue exceeds Rs. 20,000.

```
SELECT category, SUM(amount) AS revenue
FROM sales
GROUP BY category
HAVING SUM(amount) > 20000
ORDER BY revenue DESC;
```

Result:

category	revenue
----------	---------

Service	178000
Marketing	26500

Insight: Only Service and Marketing clear the Rs. 20k threshold — Design and Analytics need scaling.

Q9. Percentage contribution per category

Purpose: Advanced use of aggregation to compute share of total.

```
SELECT category,
       SUM(amount) AS revenue,
       ROUND(100.0 * SUM(amount) /
            (SELECT SUM(amount) FROM sales), 2) AS pct_share
FROM sales
GROUP BY category
ORDER BY pct_share DESC;
```

Result:

category	revenue	pct_share
Service	178000	75.96
Marketing	26500	11.3
Design	17500	7.47
Analytics	12000	5.12

Insight: Services contribute ~76% of revenue — the business is heavily service-led and should diversify to reduce concentration risk.

Q10. Monthly revenue trend (ORDER BY)

Purpose: Bucket sales by month to expose growth trajectory.

```
SELECT strftime('%Y-%m', sale_date) AS month,
       SUM(amount) AS monthly_revenue
FROM sales
GROUP BY month
ORDER BY month;
```

Result:

month	monthly_revenue
2025-01	43000
2025-02	32500
2025-03	62000
2025-04	42000
2025-05	54500

Insight: Revenue peaks in March and May, driven by large Service deals — indicating a strong Q1/Q2 pipeline.

5. Key Findings

- Tech dominates in both headcount and salary spend.
- Services generate ~76% of revenue — a concentration risk.
- Ananya Iyer is the top individual revenue driver.
- March and May are the highest-revenue months.
- Marketing has strong deal frequency but low deal size — an upsell opportunity.

6. Skills Demonstrated

SELECT projections • WHERE filtering • LIKE pattern matching • GROUP BY bucketing • COUNT / SUM / AVG / MIN / MAX aggregation • HAVING post-aggregation filters • JOIN across relations • Subqueries for percentage share • ORDER BY presentation • Understanding of SQL logical execution order (FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY).

7. Conclusion

This project proves the ability to translate a business question into a precise declarative SQL command and interpret the result set as executive-level insight. By writing queries that filter early, aggregate correctly, and respect the engine's execution order, raw data has been transformed into decisions the business can act on — the exact promise of Project 3.